

Documentação do portfólio — Daniel Coutinho Neto

1. Objetivo

Este projeto apresenta o portfólio profissional de Daniel Coutinho Neto para oportunidades de desenvolvimento back-end, Full Stack .NET e aplicações corporativas.

O site foi construído para demonstrar:

- experiência com C#, .NET, ASP.NET Core e ASP.NET Framework;
- desenvolvimento e consumo de APIs REST;
- SQL Server, ADO.NET e Entity Framework Core;
- organização arquitetural e aplicação de princípios SOLID;
- experiência com sustentação, troubleshooting e áreas de negócio;
- projetos técnicos e trajetória profissional.

O posicionamento principal utilizado no site é:

Desenvolvedor .NET Full Stack com foco em back-end.

O conteúdo informa mais de quatro anos de experiência prática com .NET, incluindo dois anos de desenvolvimento .NET em ambiente corporativo.

2. Inventário técnico e versões

Versões definidas no projeto e confirmadas no ambiente local em 29/07/2026:

Componente	Versão	Responsabilidade
Node.js	24.15.0	Ambiente de build e execução das ferramentas
npm	11.12.1	Instalação e auditoria de dependências
Next.js	16.2.12	Framework, App Router e exportação estática
React	19.2.8	Componentes e interações da interface
React DOM	19.2.8	Renderização dos componentes no navegador
PostCSS	8.5.22	Processamento de CSS durante o build
Sharp	0.35.3	Dependência de processamento de imagens do Next.js
Git	2.45.2.windows.1	Versionamento local

Componente	Versão	Responsabilidade
package-lock	formato 3	Reprodutibilidade das dependências npm

O requisito mínimo declarado para executar o projeto é Node.js 20.9.0. O workflow de publicação utiliza Node.js 24. As versões ficam fixadas em `package.json` e `package-lock.json`. Isso reduz diferenças entre o ambiente local e o pipeline.

2.1 Linguagens e formatos utilizados

Linguagem ou formato	Uso no projeto
JavaScript ECMAScript Modules	Configuração, componentes e scripts
JSX	Estrutura dos componentes React
HTML5	Documento estático gerado pelo Next.js
CSS3	Identidade visual, temas e responsividade
SVG	Ícones, favicon e imagem de compartilhamento
JSON	Manifesto npm, lockfile e configuração de hospedagem
JSON-LD	Dados estruturados do profissional para mecanismos de busca
YAML	Workflow de integração e publicação no GitHub Actions
Markdown	README e documentação técnica
PowerShell	Comandos documentados para ambiente Windows

O código-fonte utiliza JavaScript, não TypeScript. Mesmo assim, o Next.js executa sua etapa de verificação durante o build.

2.2 Dependências diretas

As únicas dependências diretas de produção são:

```
{
  "next": "16.2.12",
  "react": "19.2.8",
  "react-dom": "19.2.8"
}
```

Não são utilizadas bibliotecas de componentes, frameworks CSS, bibliotecas de ícones, sistemas de analytics ou fontes carregadas de terceiros.

2.3 Substituições de segurança

O `package.json` utiliza `overrides` para manter dependências transitivas em versões corrigidas:

```
{
  "overrides": {
    "next": {
      "postcss": "8.5.22",
      "sharp": "0.35.3"
    }
  }
}
```

Essas substituições foram adicionadas após auditoria das versões transitivas originais. Depois da atualização, a instalação npm informou zero vulnerabilidades conhecidas.

Não remova os `overrides` sem executar novamente:

```
npm.cmd audit
npm.cmd run build
```

2.4 Versões e integridade das GitHub Actions

As Actions são fixadas pelo SHA completo do commit. O comentário ao lado do SHA registra a versão legível, mas a execução utiliza a referência imutável:

Action	Versão	SHA fixado
<code>actions/checkout</code>	v6	<code>d23441a48e516b6c34aea4fa41551a30e30af803</code>
<code>actions/setup-node</code>	v6	<code>249970729cb0ef3589644e2896645e5dc5ba9c38</code>
<code>actions/configure-pages</code>	v5	<code>983d7736d9b0ae728b81ab479565c72886d7745b</code>
<code>actions/upload-pages-artifact</code>	v3	<code>56afc609e74202658d3ffba0e8f6dda462b719fa</code>
<code>actions/deploy-pages</code>	v4	<code>d6db90164ac5ed86f2b6aed7e0febac5b3c0c03e</code>
<code>github/codeql-action</code>	v4	<code>adfd868f108ac4222129de456ea554034a27db7</code>

Antes de atualizar uma Action, consulte sua documentação oficial, revise mudanças incompatíveis, troque o SHA pelo commit oficial da nova versão e execute o workflow em uma branch de validação. Nunca substitua o SHA apenas por uma tag como `@v6`.

2.5 Arquitetura de execução

O portfólio é uma aplicação Next.js com App Router e exportação totalmente estática:

```
Código Next.js e React
  ↓
next build
  ↓
HTML + CSS + JavaScript em out/
  ↓
Artefato do GitHub Actions
  ↓
GitHub Pages
```

O site publicado não precisa de servidor Node.js, banco de dados ou API. O navegador recebe arquivos estáticos.

2.6 Divisão entre servidor e cliente

O conteúdo principal em `app/page.jsx` é renderizado estaticamente durante o build.

Somente os recursos que dependem do navegador são componentes de cliente:

- `SiteHeader.jsx`: menu, tema e `localStorage`;
- `CopyEmailButton.jsx`: acesso à área de transferência;
- `ClientEffects.jsx`: `IntersectionObserver` e animações de entrada.

Essa divisão reduz JavaScript enviado ao navegador e preserva a indexação do conteúdo.

2.7 Estado e armazenamento

O projeto não possui autenticação, sessão, cookies ou banco de dados.

A única preferência persistida é o tema visual:

```
Chave: portfolio-theme
Armazenamento: localStorage
Valores: dark ou light
```

2.8 Compatibilidade e acessibilidade

O site foi projetado para navegadores modernos com suporte a:

- CSS Custom Properties;
- CSS Grid e Flexbox;
- `IntersectionObserver`;
- `localStorage`;
- Clipboard API.

Há fallback para o contato por `mailto` quando a Clipboard API não está disponível.

Recursos de acessibilidade implementados:

- idioma `pt-BR`;
- link para pular ao conteúdo;
- navegação semântica;

- textos alternativos;
- rótulos ARIA em controles;
- foco visível;
- respeito a `prefers-reduced-motion`;
- temas claro e escuro;
- layout adaptável a telas menores.

2.9 Decisões técnicas

- Exportação estática para hospedagem simples e baixo custo operacional.
- Componentes React apenas onde há benefício real.
- CSS próprio para evitar peso e dependências visuais.
- SVG próprio para evitar requisições externas.
- Sem formulário com back-end; o contato utiliza e-mail e perfis profissionais.
- Sem telefone público para reduzir exposição de dados pessoais.
- Sem currículo em PDF público nesta versão.
- Sem métricas inventadas ou barras percentuais de conhecimento.

3. Requisitos

Para executar e modificar o projeto:

- Node.js 20.9 ou superior;
- npm;
- editor de código, preferencialmente Visual Studio Code;
- navegador atualizado.

Para confirmar as versões instaladas:

```
node --version  
npm.cmd --version
```

4. Executar localmente

Abra um terminal na pasta do projeto:

```
cd C:\Users\danie.000\source\repos\Portifolio
```

Instale as dependências na primeira execução:

```
npm.cmd install
```

Inicie o ambiente local:

```
npm.cmd run dev
```

Acesse:

```
http://localhost:3000
```

Enquanto o servidor estiver ativo, alterações salvas no código aparecerão automaticamente no navegador.

Para interromper o servidor, pressione **Ctrl+C** no terminal em que ele está executando.

5. Estrutura principal

```
Portifolio/
├── .github/
│   ├── workflows/
│   │   ├── deploy-pages.yml
│   │   └── security.yml
│   ├── CODEOWNERS
│   └── dependabot.yml
├── .openai/
│   └── hosting.json
├── app/
│   ├── globals.css
│   ├── layout.jsx
│   ├── not-found.jsx
│   └── page.jsx
├── components/
│   ├── ClientEffects.jsx
│   ├── CopyEmailButton.jsx
│   ├── Icon.jsx
│   └── SiteHeader.jsx
├── docs/
│   ├── documenteacao-site.md
│   └── documenteacao-site.pdf
├── public/
│   ├── favicon.svg
│   └── og-card.svg
├── scripts/
│   ├── check-content.mjs
│   ├── generate-docs-pdf.mjs
│   └── security-check.mjs
├── .gitattributes
├── AGENTS.md
├── next.config.mjs
├── package.json
└── README.md
```

As pastas `.next`, `out` e `node_modules` são geradas automaticamente e não devem ser editadas manualmente.

O arquivo `.gitattributes` mantém os arquivos de texto com final de linha LF no repositório, inclusive os workflows executados em Linux, e marca o PDF como binário para impedir conversões acidentais.

6. Alterar textos e informações profissionais

O conteúdo principal está em:

```
app/page.jsx
```

Nesse arquivo estão:

- apresentação principal;
- indicadores da trajetória;
- resumo profissional;
- projetos;
- competências;
- experiências;
- formação e certificações;
- idiomas;
- contatos.

Para localizar rapidamente uma seção no Visual Studio Code, utilize `Ctrl+F`.

6.1 Apresentação principal

Procure por:

```
<section className="hero section" id="inicio">
```

Nesse bloco é possível alterar:

- título principal;
- especialidade;
- descrição profissional;
- botões;
- indicadores de experiência.

Ao modificar o tempo de experiência, preserve a distinção entre experiência prática total com .NET e experiência formal em ambiente corporativo.

6.2 Resumo profissional

Procure por:

```
<section className="section about-section" id="sobre">
```

Atualize os parágrafos sempre que houver mudança relevante no posicionamento, objetivo profissional ou trajetória.

6.3 Contatos

No início de `app/page.jsx` estão estas constantes:

```
const githubUrl = "https://github.com/danielcouthoneto";
const linkedinUrl = "https://www.linkedin.com/in/daniel-couthoneto";
const email = "danielcouthoneto@outlook.com";
```

Altere-as caso algum endereço mude.

O telefone não está publicado no site para reduzir exposição de informação pessoal e mensagens indesejadas.

7. Adicionar ou alterar projetos

Os projetos estão no array:

```
const projects = [
  // projetos
];
```

Para adicionar um projeto, inclua um objeto com esta estrutura:

```
{
  id: "identificador-unico",
  order: "04",
  label: "Categoria",
  title: "Nome do projeto",
  subtitle: "Resumo curto",
  description:
    "Descrição geral do projeto e do valor entregue.",
  challenge:
    "Problema ou necessidade que originou o projeto.",
  solution:
    "Decisões técnicas adotadas para resolver o problema.",
  tags: [
    ".NET 8",
    "ASP.NET Core",
    "SQL Server"
  ]
}
```

Regras importantes:

- `id` não pode se repetir;
- use apenas letras minúsculas, números e hífen no `id`;
- mantenha a numeração de `order`;
- coloque vírgula entre os projetos;
- liste apenas tecnologias realmente utilizadas;

- descreva decisões, não somente funcionalidades;
- não invente métricas ou resultados.

Para destacar um projeto como principal, adicione:

```
featured: true
```

Evite manter mais de um projeto principal ao mesmo tempo.

8. Adicionar experiência profissional

As experiências estão no array:

```
const experiences = [  
  // experiências  
];
```

Exemplo:

```
{  
  period: "abr/2026 – atual",  
  role: "Desenvolvedor .NET",  
  company: "Nome da empresa",  
  summary: "Resumo da responsabilidade principal.",  
  highlights: [  
    "Primeiro resultado ou responsabilidade.",  
    "Segundo resultado ou responsabilidade.",  
    "Tecnologias e colaboração envolvidas."  
  ],  
  accent: true  
}
```

Utilize `accent: true` somente na experiência que deve receber destaque visual.

Não publique informações confidenciais, código privado, nomes de clientes sem autorização ou indicadores internos da empresa.

9. Alterar competências

As competências estão no array:

```
const skillGroups = [  
  // grupos  
];
```

Cada grupo possui:

- `icon`: ícone apresentado;

- `title`: nome do grupo;
- `description`: objetivo daquele conjunto;
- `skills`: tecnologias e práticas.

O site separa a narrativa das competências em três níveis:

1. experiência aplicada profissionalmente;
2. aplicação em projetos;
3. evolução contínua.

Mantenha essa distinção para que o portfólio não apresente como experiência profissional algo utilizado apenas em estudos.

10. Alterar formação, certificações e idiomas

Esses dados ficam na seção:

```
<section className="section education-section" id="formacao">
```

Ao adicionar uma certificação, informe:

- ano;
- nome oficial;
- instituição responsável.

Não é necessário listar cursos muito básicos quando eles deixarem de agregar valor ao posicionamento profissional.

11. Cores, fontes e layout

Os estilos estão em:

```
app/globals.css
```

As principais cores são variáveis no início do arquivo:

```
:root {  
  --bg: #08111f;  
  --surface: #0d1929;  
  --text: #f2f6fb;  
  --accent: #8f7cff;  
  --cyan: #53d8e7;  
}
```

O tema claro está em:

```
:root[data-theme="light"] {  
  /* cores do tema claro */  
}
```

Ao alterar uma cor, verifique os dois temas e mantenha contraste suficiente entre texto e fundo.

Os principais pontos de responsividade estão nas regras:

```
@media (max-width: 1020px)  
@media (max-width: 860px)  
@media (max-width: 680px)
```

12. Componentes interativos

Cabeçalho e troca de tema

Arquivo:

```
components/SiteHeader.jsx
```

Responsável por:

- navegação principal;
- menu para telas menores;
- tema claro e escuro;
- armazenamento da preferência no navegador.

Botão de copiar e-mail

Arquivo:

```
components/CopyEmailButton.jsx
```

Animações de entrada

Arquivo:

```
components/ClientEffects.jsx
```

As animações respeitam a preferência de acessibilidade `prefers-reduced-motion`.

Ícones

Arquivo:

```
components/Icon.jsx
```

Os ícones são vetoriais e não dependem de serviços externos.

13. Adicionar imagens

Coloque imagens dentro de `public`.

Exemplo:

```
public/projetos/sisageli-login.webp
```

No componente:

```

```

Recomendações:

- prefira WebP, AVIF ou SVG;
- remova dados pessoais de screenshots;
- use nomes de arquivo sem espaços e sem acentos;
- escreva texto alternativo que explique a imagem;
- evite imagens muito pesadas;
- não publique credenciais, URLs internas ou dados reais.

14. SEO e compartilhamento

Título, descrição, palavras-chave e dados de compartilhamento ficam em:

```
app/layout.jsx
```

A imagem utilizada quando o site é compartilhado está em:

```
public/og-card.svg
```

Os dados estruturados profissionais ficam no objeto `personSchema`, em `app/page.jsx`.

Ao alterar nome, especialidade, endereço do site ou contatos, revise:

- `app/layout.jsx`;
- `personSchema`;
- `public/og-card.svg`;
- esta documentação.

15. Validar alterações

Execute a validação de conteúdo:

```
npm.cmd run check
```

Gere o build de produção:

```
npm.cmd run build
```

Valide o código, os workflows e o artefato gerado:

```
npm.cmd run security:check
```

Audite vulnerabilidades conhecidas nas dependências:

```
npm.cmd audit --omit=dev --audit-level=high
```

O resultado estático será criado em:

```
out/
```

Uma alteração está pronta quando:

- a validação termina sem erros;
- o build termina sem erros;
- a auditoria npm não encontra vulnerabilidades altas ou críticas;
- a verificação de segurança não encontra segredos ou controles ausentes;
- o site abre em desktop e celular;
- os links funcionam;
- os dois temas continuam legíveis;
- não existem informações provisórias;
- a documentação foi atualizada.

16. Atualizar esta documentação

Qualquer alteração funcional, visual, estrutural ou de conteúdo no site deve atualizar:

```
docs/documenteacao-site.md  
docs/documenteacao-site.pdf
```

Primeiro altere este arquivo Markdown. Depois gere novamente o PDF:

```
npm.cmd run docs:pdf
```

O PDF é gerado a partir do Markdown para evitar versões divergentes.

Depois da geração, confirme a data e o conteúdo do PDF.

16.1 Controles automáticos de sincronização

O arquivo `AGENTS.md` registra para futuras manutenções a obrigação de atualizar esta documentação sempre que o site mudar.

O comando:

```
npm.cmd run check
```

também verifica:

- presença do Markdown e do PDF;
- cabeçalho válido do arquivo PDF;
- se o PDF foi gerado depois da última alteração do Markdown;
- presença das versões das dependências diretas nesta documentação;
- arquivos essenciais do projeto;
- âncoras internas;
- ausência de textos provisórios;
- presença dos principais dados profissionais.

Se a verificação informar que o PDF está desatualizado:

```
npm.cmd run docs:pdf  
npm.cmd run check
```

16.2 Como o PDF é produzido

O script:

```
scripts/generate-docs-pdf.mjs
```

executa as seguintes etapas:

1. lê `docs/documentacao-site.md`;
2. converte títulos, listas, tabelas, links e blocos de código para HTML;
3. aplica estilos próprios para papel A4;
4. abre Google Chrome ou, como fallback, Microsoft Edge em modo invisível e

com rasterização gráfica desativada;

1. imprime o HTML como `docs/documentacao-site.pdf`;
2. remove os arquivos temporários.

O script procura primeiro pelo Google Chrome e depois pelo Microsoft Edge nos caminhos padrão do Windows. Essa ordem evita uma incompatibilidade observada entre algumas versões do Edge headless e o processo gráfico no Windows.

17. Publicação no GitHub Pages

O workflow está em:

```
.github/workflows/deploy-pages.yml
```

Quando a publicação for autorizada:

1. atualize esta documentação;
2. gere o PDF;
3. valide o conteúdo;
4. execute o build;
5. execute a auditoria e a verificação de segurança;
6. revise os arquivos que entrarão no commit;
7. crie o commit;
8. envie a branch `main` ao GitHub;
9. configure **Settings > Pages > GitHub Actions** no repositório;
10. aguarde os workflows de publicação e segurança;
11. confirme HTTPS, links, arquivos carregados e resultado das verificações.

O workflow executará:

1. checkout de uma referência conhecida;
2. leitura dos metadados do GitHub Pages;
3. instalação reproduzível com `npm ci --ignore-scripts`;
4. auditoria das dependências de produção;
5. validação do conteúdo;
6. build estático;
7. validação de segurança do artefato;
8. envio exclusivo da pasta `out`;
9. publicação no GitHub Pages com identidade temporária do GitHub Actions.

A Action `configure-pages` é utilizada sem o parâmetro `static_site_generator: next`. O `next.config.mjs` já calcula `basePath` e `assetPrefix` a partir de `GITHUB_REPOSITORY`; permitir que a Action também reescreva a configuração causaria duas fontes de configuração e é incompatível com a sintaxe utilizada pelo Next.js 16.

O build possui somente `contents: read` e `pages: read`. O deploy, executado em outro job, possui somente `pages: write` e `id-token: write`. Essas permissões não são incorporadas ao HTML, CSS ou JavaScript publicado.

Os workflows definem `NEXT_TELEMETRY_DISABLED=1`. Isso impede o envio de telemetria anônima do Next.js durante os builds do GitHub Actions; não existe analytics nem telemetria no navegador do visitante.

O endereço público esperado para este repositório é:

```
https://danielcouthoneto.github.io/portfolio/
```

Nenhum commit ou deploy deve ser feito antes da revisão e autorização do proprietário.

18. Segurança

18.1 Modelo de risco

O portfólio público é composto somente por HTML, CSS, JavaScript e SVG. Ele não possui:

- autenticação ou sessão;
- banco de dados;
- API própria;
- formulário enviado a um servidor;
- campo de senha;
- token do GitHub;
- chave de API;
- arquivo `.env`;
- analytics ou script de terceiros;
- conteúdo privado do currículo;
- telefone público.

O link para o GitHub é uma navegação comum para o perfil público. Ele não autoriza o site a ler repositórios privados, alterar configurações, criar commits ou acessar a conta.

O arquivo `.openai/hosting.json` contém somente o identificador opaco do projeto de hospedagem conectado. Ele não contém senha ou token de acesso.

18.2 Proteções no navegador

O HTML de produção inclui uma Content Security Policy com:

```
default-src 'self'  
script-src 'self' 'unsafe-inline'  
style-src 'self' 'unsafe-inline'  
img-src 'self' data:  
font-src 'self'  
connect-src 'self'  
object-src 'none'  
base-uri 'self'  
form-action 'self'  
upgrade-insecure-requests
```

`unsafe-inline` é necessário para o código de inicialização gerado pelo Next.js na exportação estática. O risco é reduzido porque não existe entrada de usuário, HTML dinâmico ou conteúdo carregado de terceiros.

A política de referência é `strict-origin-when-cross-origin`, limitando detalhes do endereço enviados ao navegar para outra origem. O GitHub Pages utiliza HTTPS e a publicação deve manter a opção de exigir HTTPS habilitada.

18.3 Verificação automática local

O script `scripts/security-check.mjs` verifica:

- tokens conhecidos de GitHub, AWS, Stripe e Slack;

- marcadores de chave privada;
- arquivos `.env`, `.key`, `.pem`, `.pfx` e `.p12`;
- Actions sem SHA completo de 40 caracteres;
- bloqueio das permissões padrão dos workflows;
- permissões necessárias do pipeline de Pages;
- presença da CSP e da política de referência no HTML gerado;
- recursos ativos usando HTTP;
- formulários, campos de senha ou chamadas à API do GitHub no artefato;
- existência do artefato estático.

Execute sempre depois do build:

```
npm.cmd run build
npm.cmd run security:check
```

18.4 Proteções no GitHub

Os arquivos de segurança são:

Arquivo	Finalidade
<code>.github/workflows/deploy-pages.yml</code>	Build e publicação com permissões separadas
<code>.github/workflows/security.yml</code>	Auditoria, build, verificação própria e CodeQL
<code>.github/dependabot.yml</code>	Atualizações semanais de npm e GitHub Actions
<code>.github/CODEOWNERS</code>	Identificação do responsável por arquivos críticos

O workflow de segurança é executado em `push`, pull request, manualmente e toda segunda-feira. O CodeQL analisa JavaScript/TypeScript. O Dependabot abre pull requests de atualização, mas nenhuma atualização deve ser aceita sem revisar o build e as mudanças.

Para o repositório público, mantenha habilitados:

- alertas de dependências vulneráveis;
- atualizações automáticas de segurança;
- secret scanning e push protection;
- permissões padrão de workflows em somente leitura;
- bloqueio de force push e exclusão da branch `main`;
- HTTPS obrigatório no GitHub Pages.

O `GITHUB_TOKEN` usado pelo Actions é criado para o job, limitado ao repositório e expira ao final da execução. O token não deve ser salvo em arquivo, variável `NEXT_PUBLIC_*`, log ou conteúdo de `out`.

18.5 Limites da garantia

Uma auditoria reduz riscos conhecidos no momento da publicação, mas nenhum site pode ser declarado invulnerável para sempre. Novas falhas podem ser descobertas nas dependências ou na plataforma. Por isso,

acompanhe os alertas do GitHub, revise as atualizações do Dependabot e execute as verificações após cada alteração.

Se futuramente forem adicionados formulário, API, analytics, domínio personalizado ou serviço externo, o modelo de risco muda. Faça nova revisão antes de publicar. Para domínio personalizado, valide DNS e propriedade para evitar configuração abandonada e risco de tomada de domínio.

19. Problemas comuns

O comando `npm` é bloqueado no PowerShell

Utilize:

```
npm.cmd run dev
```

em vez de:

```
npm run dev
```

A porta 3000 está ocupada

Execute em outra porta:

```
npm.cmd run dev -- -p 3001
```

Depois acesse:

```
http://localhost:3001
```

O build apresenta erro de sintaxe

Verifique:

- vírgulas entre itens dos arrays;
- aspas abertas e fechadas;
- parênteses e chaves;
- fechamento das tags JSX;
- itens duplicados.

Uma imagem não aparece

Confirme:

- se está dentro de `public`;
- se o nome está correto;
- se maiúsculas e minúsculas coincidem;
- se o caminho começa com `/`;
- se o arquivo foi incluído no projeto.

20. Checklist para futuras alterações

- ☐ Alterar o conteúdo ou código necessário.
- ☐ Revisar português, datas e informações profissionais.
- ☐ Não exagerar senioridade ou domínio técnico.
- ☐ Não publicar dados confidenciais.
- ☐ Testar em `http://localhost:3000`.
- ☐ Testar tema claro e escuro.
- ☐ Testar navegação em tela pequena.
- ☐ Executar `npm.cmd run check`.
- ☐ Executar `npm.cmd run build`.
- ☐ Executar `npm.cmd audit --omit=dev --audit-level=high`.
- ☐ Executar `npm.cmd run security:check`.
- ☐ Confirmar que nenhum token, senha, `.env` ou chave foi adicionado.
- ☐ Atualizar `docs/documentacao-site.md`.
- ☐ Executar `npm.cmd run docs:pdf`.
- ☐ Revisar `docs/documentacao-site.pdf`.
- ☐ Conferir os workflows e alertas de segurança do GitHub.
- ☐ Após publicar, confirmar HTTPS e o sucesso do GitHub Actions.
- ☐ Criar commit somente após aprovação.

Documento mantido junto ao código-fonte do portfólio.